



S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Vansh Khapre

USN: CD24060

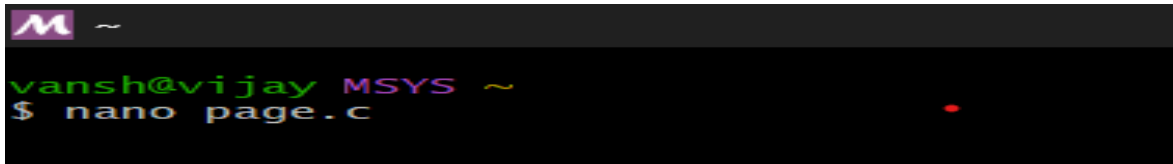
Semester / Year: 4th SEM / 2nd YEAR

Academic Session: 2025-26

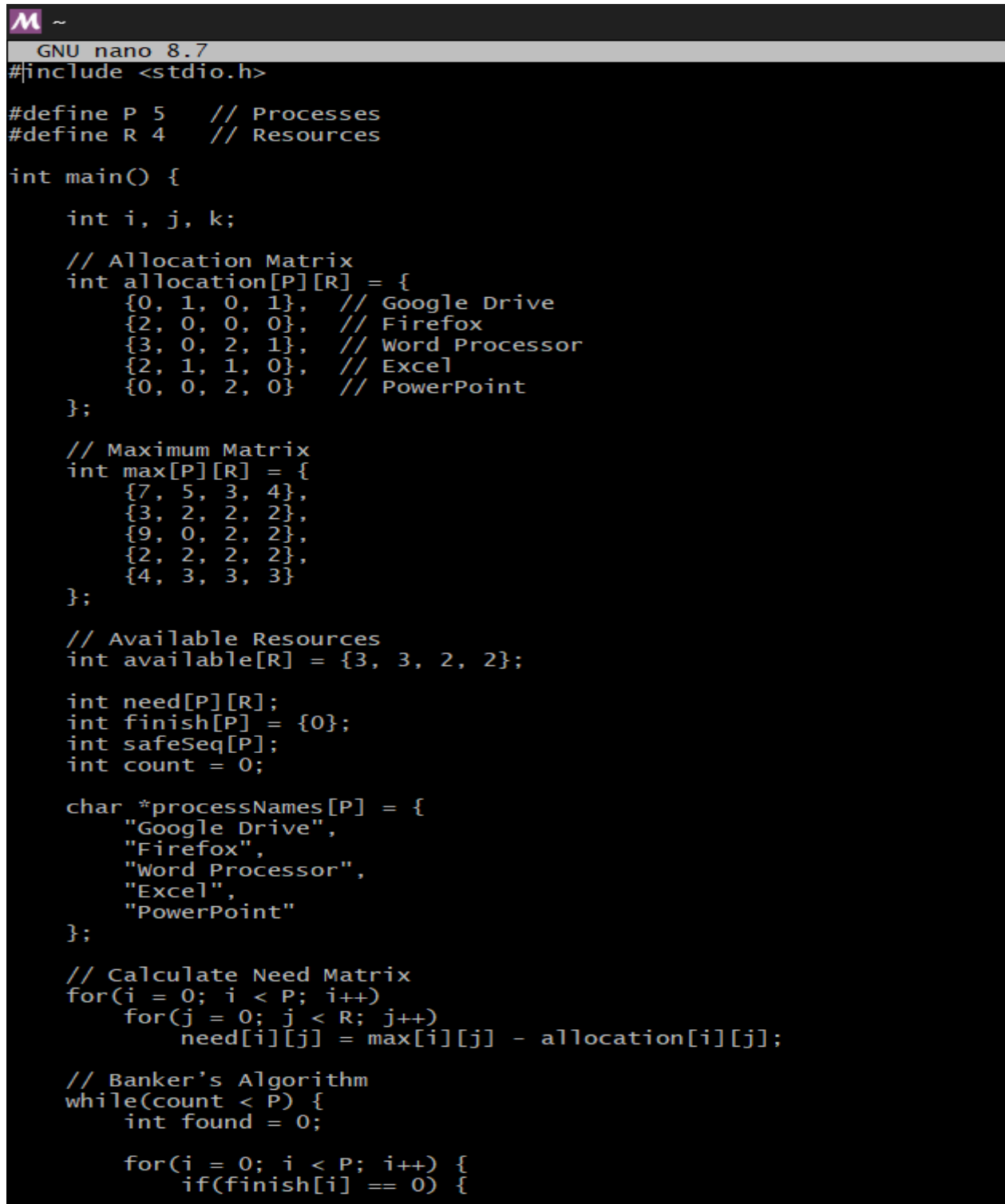
Date of Performance:

Date of Submission:

❖ CODE :



```
M ~  
vansh@vijay MSYS ~  
$ nano page.c
```



```
M ~  
GNU nano 8.7  
#include <stdio.h>  
  
#define P 5    // Processes  
#define R 4    // Resources  
  
int main() {  
    int i, j, k;  
  
    // Allocation Matrix  
    int allocation[P][R] = {  
        {0, 1, 0, 1}, // Google Drive  
        {2, 0, 0, 0}, // Firefox  
        {3, 0, 2, 1}, // Word Processor  
        {2, 1, 1, 0}, // Excel  
        {0, 0, 2, 0}  // PowerPoint  
    };  
  
    // Maximum Matrix  
    int max[P][R] = {  
        {7, 5, 3, 4},  
        {3, 2, 2, 2},  
        {9, 0, 2, 2},  
        {2, 2, 2, 2},  
        {4, 3, 3, 3}  
    };  
  
    // Available Resources  
    int available[R] = {3, 3, 2, 2};  
  
    int need[P][R];  
    int finish[P] = {0};  
    int safeSeq[P];  
    int count = 0;  
  
    char *processNames[P] = {  
        "Google Drive",  
        "Firefox",  
        "Word Processor",  
        "Excel",  
        "PowerPoint"  
    };  
  
    // Calculate Need Matrix  
    for(i = 0; i < P; i++)  
        for(j = 0; j < R; j++)  
            need[i][j] = max[i][j] - allocation[i][j];  
  
    // Banker's Algorithm  
    while(count < P) {  
        int found = 0;  
  
        for(i = 0; i < P; i++) {  
            if(finish[i] == 0) {
```

```

for(i = 0; i < P; i++) {
    if(finish[i] == 0) {
        int flag = 1;

        for(j = 0; j < R; j++) {
            if(need[i][j] > available[j]) {
                flag = 0;
                break;
            }
        }

        if(flag) {
            for(k = 0; k < R; k++)
                available[k] += allocation[i][k];

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0) {
    printf("\nSystem is NOT in SAFE state.\n");
    return 0;
}

printf("\nSystem is in SAFE state.\nSafe Sequence:\n");
for(i = 0; i < P; i++)
    printf("%s -> ", processNames[safeSeq[i]]);
printf("END\n");

return 0;
}

```

❖ OUTPUT :

```

vansh@vijay MSYS ~
$ ./page

System is in SAFE state.
Safe Sequence:
Firefox -> Excel -> Word Processor -> PowerPoint -> Google Drive -> END

```